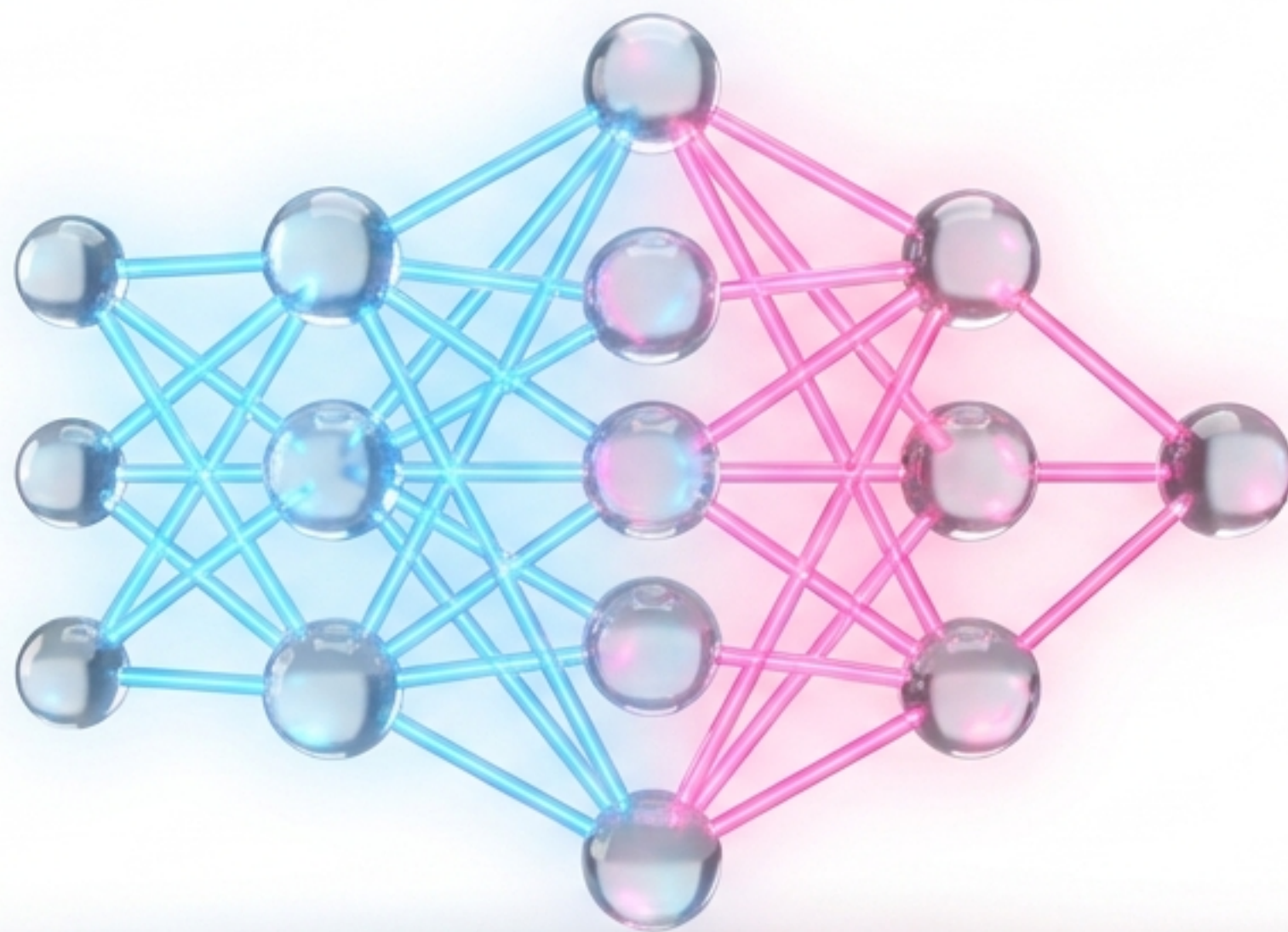


Nền tảng Toán học của Neural Networks

Giải Mã Đồ Thị Tính Toán & Lan Truyền Ngược

Hiểu cặn kẽ "phép thuật" học máy thông qua Đạo hàm tự động và Cấu trúc dữ liệu trực quan.



Dữ liệu

Lấy một lô dữ liệu (batch) gồm đầu vào (x) và mục tiêu (y_{true}).

Lan truyền xuôi

Mạng xử lý x để tạo ra dự đoán (y_{pred}).

Cập nhật Trọng số

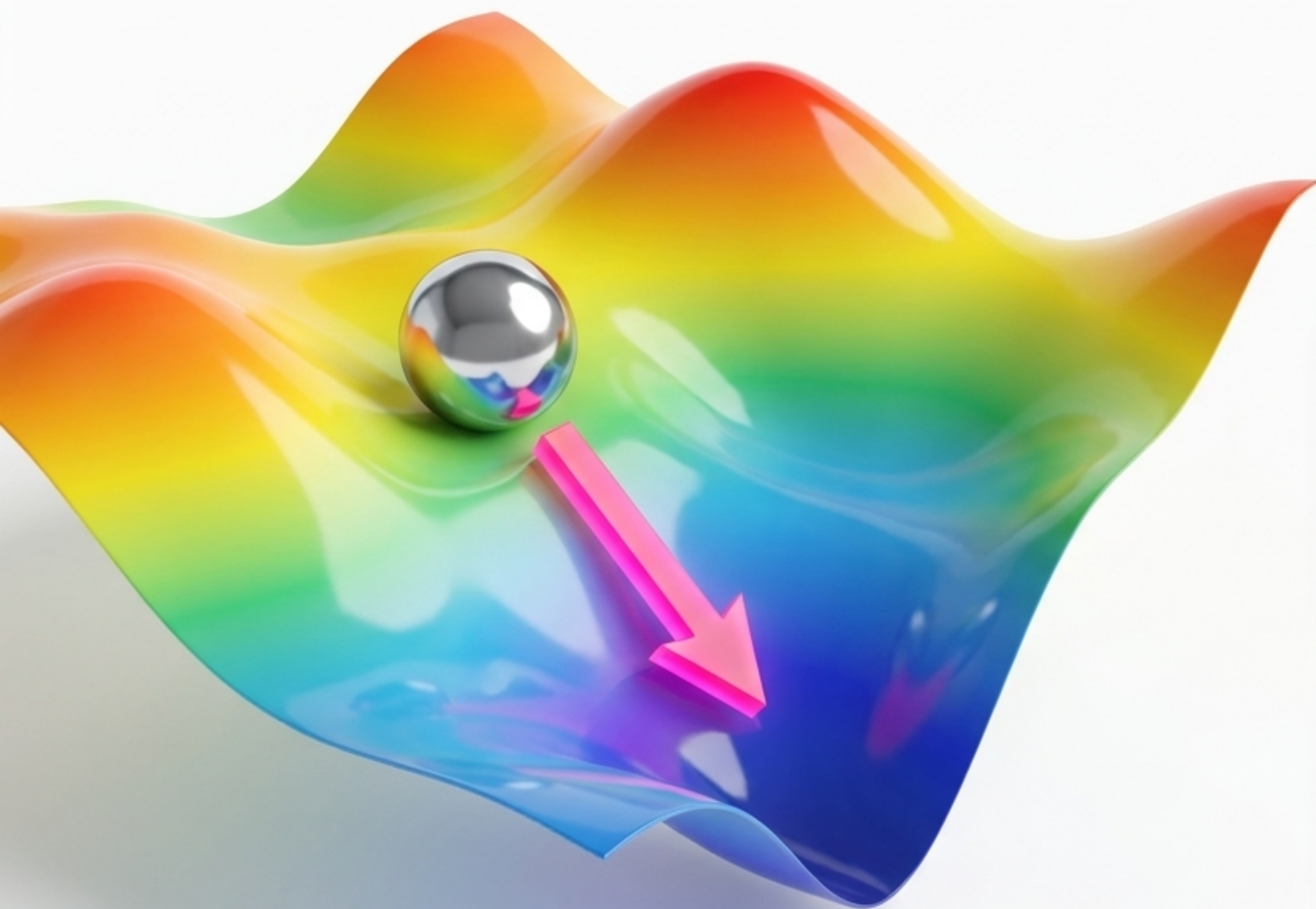
Điều chỉnh các tham số để giảm Loss.

Đo lường sai số

Tính toán Loss - khoảng cách giữa y_{pred} và y_{true} .

Vấn đề cốt lõi: Mạng có hàng triệu tham số. Làm sao biết nên tăng hay giảm từng tham số?

Giải pháp: Tối ưu hóa dựa trên Gradient.



Gradient: Bản đồ Chỉ hướng

1 Đạo hàm & Gradient

Đạo hàm cho biết một hàm số thay đổi ra sao khi đầu vào thay đổi. Gradient là phiên bản đa chiều của đạo hàm, mô tả độ cong của bề mặt Loss.

2 Quy tắc vàng

Để giảm sai số (Loss), luôn di chuyển Trọng số một bước nhỏ theo hướng ngược lại với Gradient.

Nút thắt của các Hàm phức tạp: Quy tắc Chuỗi



Thách thức

Neural Networks là một chuỗi dài các phép toán Tensor cơ bản (Cộng, Nhân, ReLU...). Làm sao tính đạo hàm của toàn bộ chuỗi khổng lồ này?

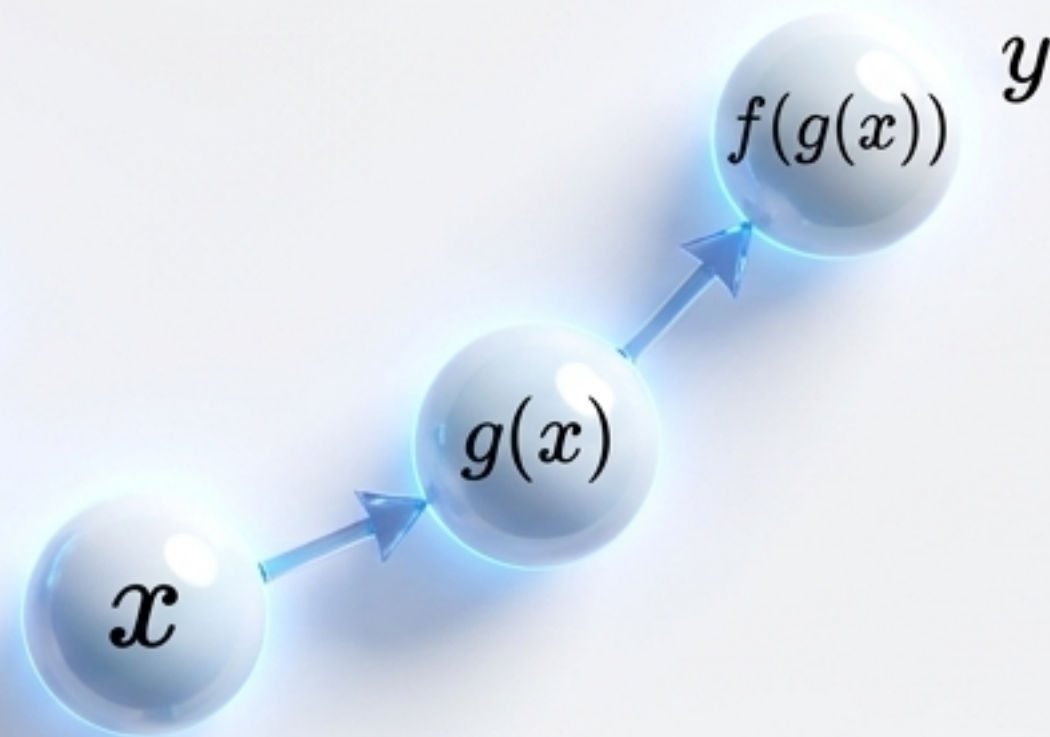
Giải pháp

Quy tắc chuỗi (Chain Rule) cho phép tính đạo hàm của một hàm hợp bằng cách nhân đạo hàm của các hàm thành phần.

$$\text{grad}(y, x) = \text{grad}(y, x_1) \times \text{grad}(x_1, x)$$

Đồ thị Tính toán: Biến Phép tính thành Dữ liệu

$$y = f(g(x))$$



Khái niệm

Đồ thị tính toán là một cấu trúc dữ liệu có hướng.

Đột phá

Thay vì viết mã toán học cứng nhắc, ta mã hóa biểu thức thành một mạng lưới các Node.

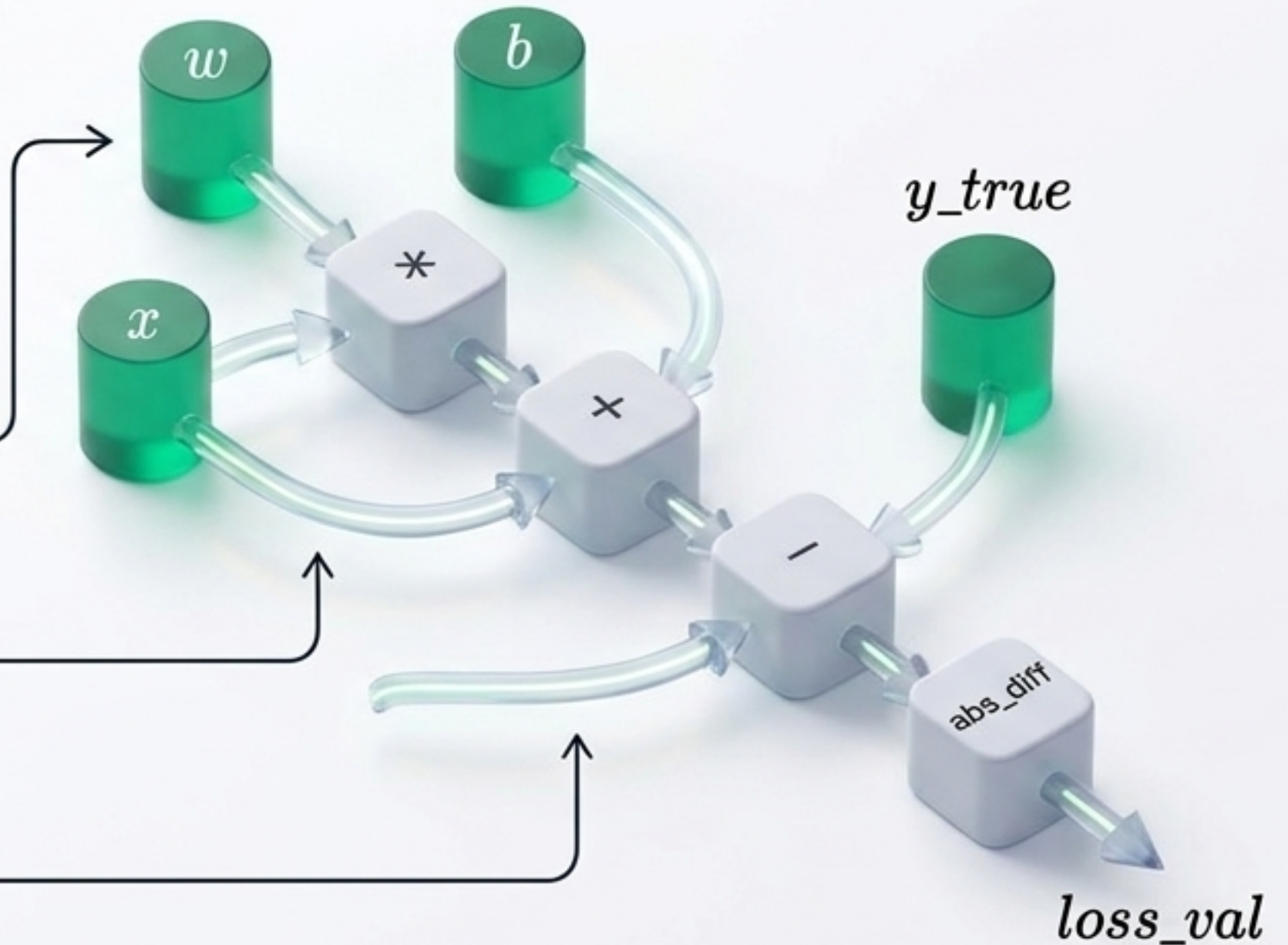
Ý nghĩa

Cho phép máy tính xử lý "phép tính như dữ liệu" — nền tảng để tự động sinh ra đạo hàm cho bất kỳ kiến trúc mạng nào.

Giải phẫu một Cấu trúc Đồ thị

```
loss_val = abs(y_true - (x * w + b))
```

- 1 **Nút Đầu vào:** Dữ liệu (x , y_{true}) & Trọng số (w , b).
- 2 **Nút Toán tử:** Các phép tính Nhân (*), Cộng (+), Sai số (abs_diff).
- 3 **Cạnh / Mũi tên:** Đường dẫn luân chuyển của Tensor từ Nút này sang Nút khác.



Thiết lập Bài toán Ví dụ

Dữ liệu thực tế

Đầu vào $x = 2$,
Mục tiêu $y_{\text{true}} = 4$.



Trọng số khởi tạo

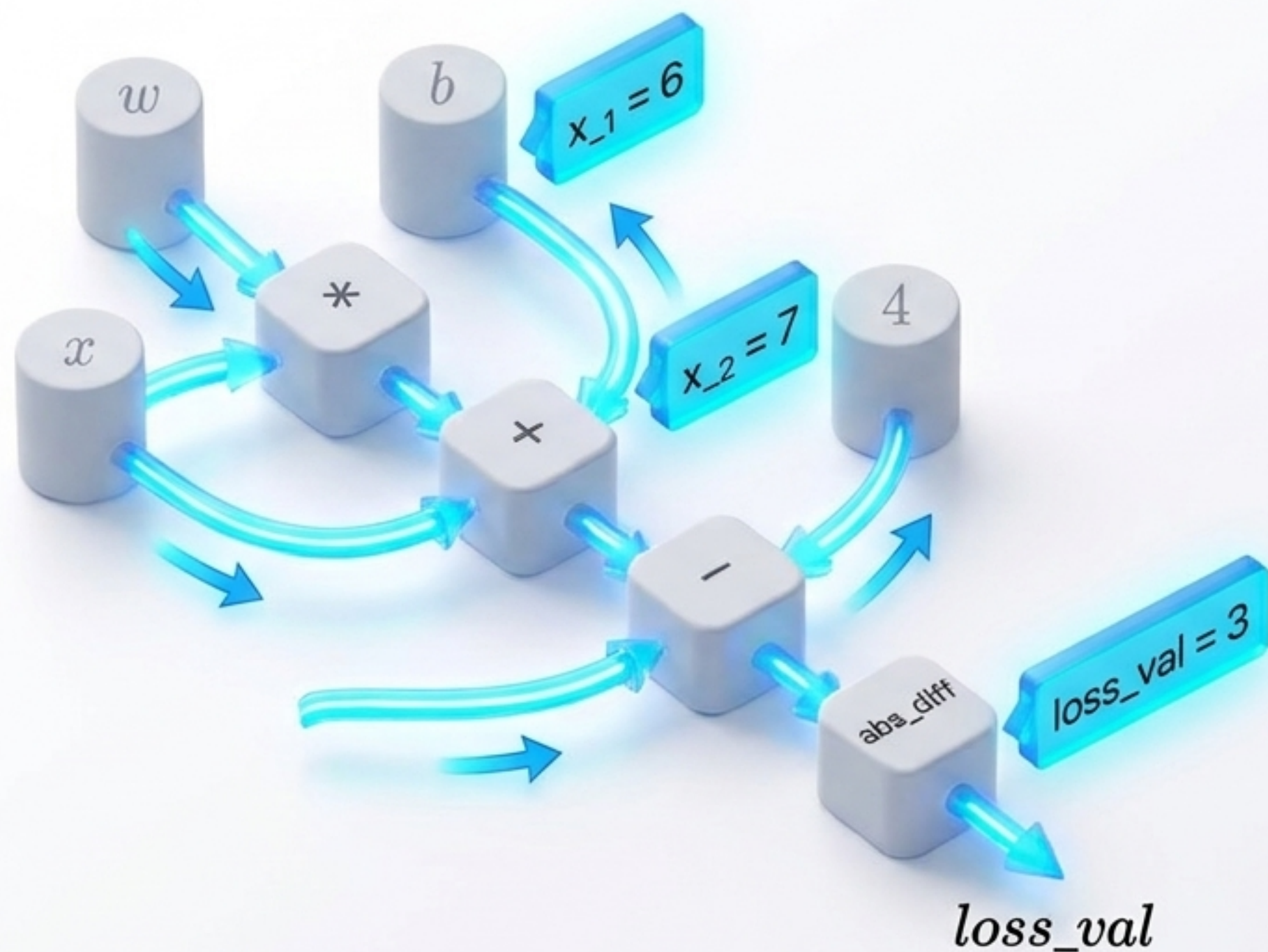
Trọng số $w = 3$,
Độ lệch $b = 1$.

Hàm Loss

Khoảng cách tuyệt đối
 $\text{abs}(y_{\text{true}} - y_{\text{pred}})$.

Mục tiêu: Cần thay đổi w và b như thế nào để Loss tiến về 0?

Bước 1: Lan truyền Xuôi (Forward Pass)



1

Nút Nhân:

$$x_1 = x \times w = 2 \times 3 = 6$$

2

Nút Cộng:

$$x_2 = x_1 + b = 6 + 1 = 7$$

(Đây chính là y_pred)

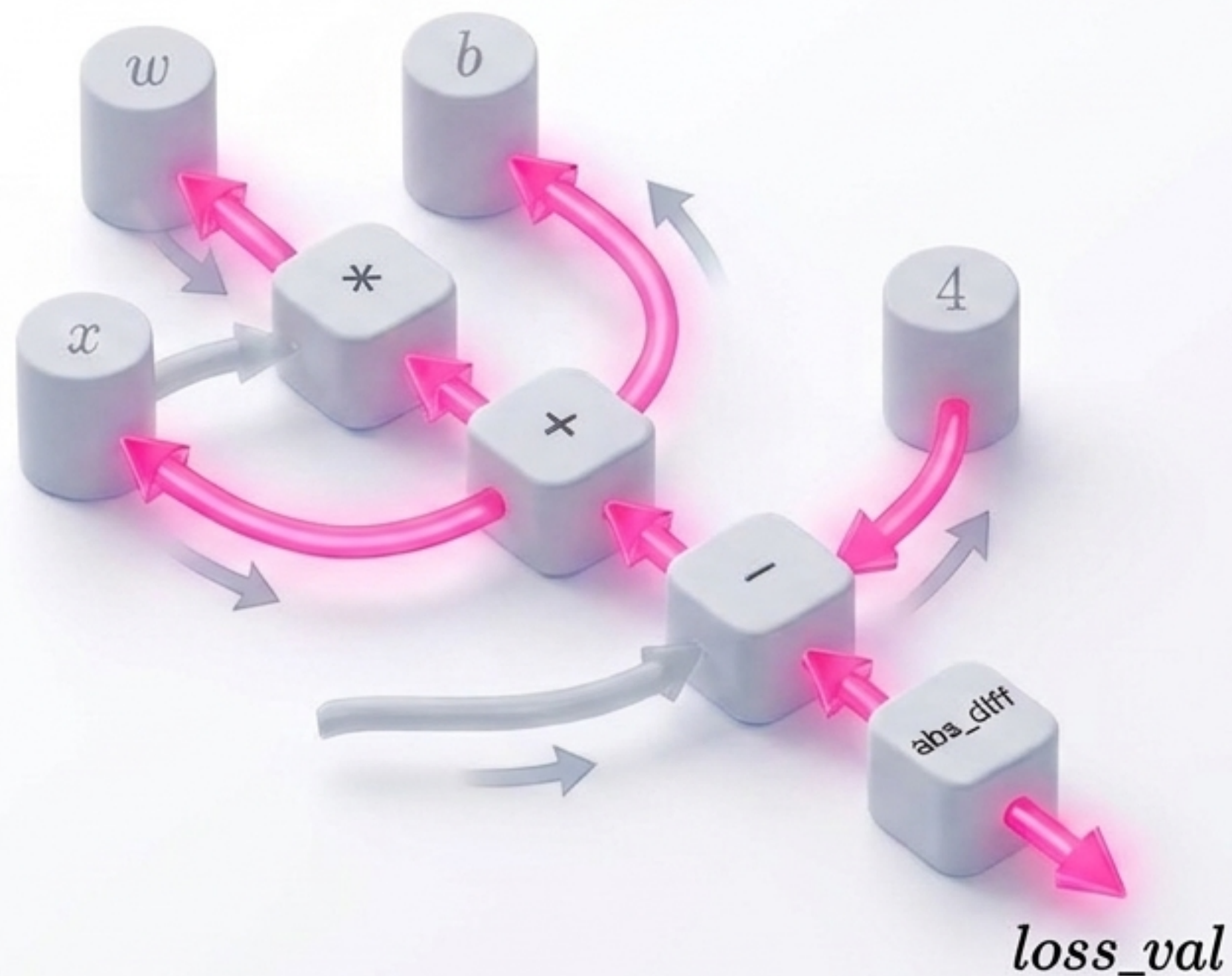
3

Nút Loss:

$$loss_val = abs(y_true - x_2)$$
$$= abs(4 - 7) = 3$$

Sai số hiện tại: **3. Quá cao!**

Đảo ngược Đồ thị: Bản chất Lan truyền Ngược



Tại mỗi cạnh kết nối Nút A đến Nút B, hãy hỏi:

Nếu A thay đổi một chút, B sẽ thay đổi bao nhiêu?

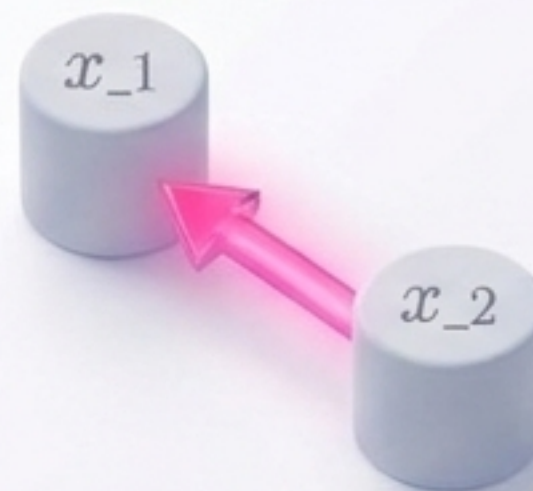
Đây chính là $\text{grad}(B, A)$. Chúng ta gắn vào mỗi cạnh đảo ngược một giá trị Đạo hàm cục bộ (Local Gradient).

Bước 2: Tính Đạo hàm Cục bộ (Local Gradients)



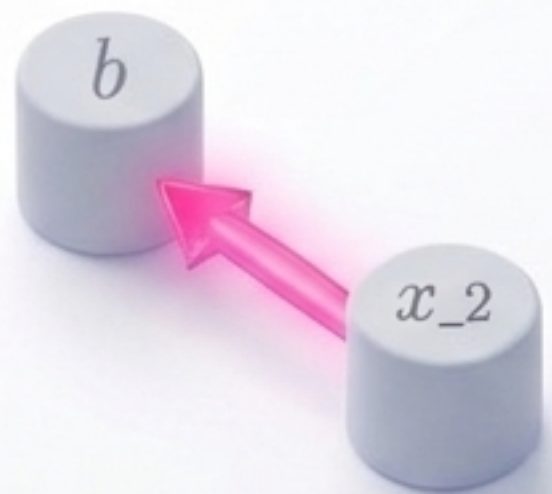
Hàm Loss

$\text{grad}(\text{loss_val}, x_2) = 1$
(Vì $\text{loss} = \text{abs}(4 - x_2)$,
 x_2 tăng 1 thì loss tăng 1)



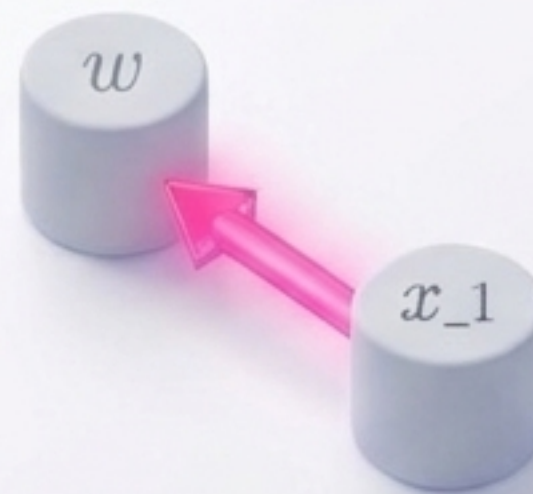
Phép Cộng 1

$\text{grad}(x_2, x_1) = 1$
(Vì $x_2 = x_1 + b$, phép
cộng có độ dốc 1)



Phép Cộng 2

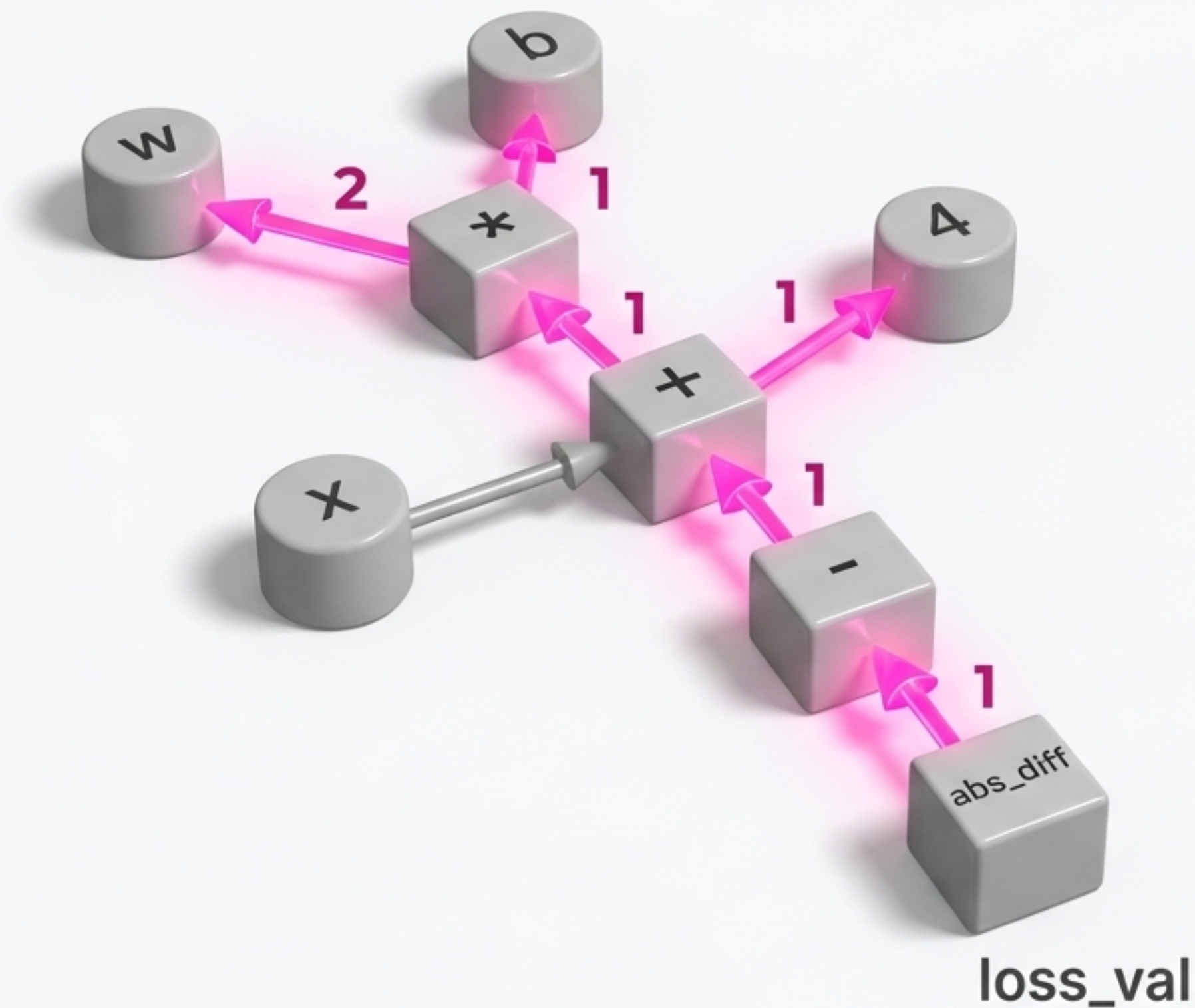
$\text{grad}(x_2, b) = 1$
(Tương tự phép cộng)



Phép Nhân

$\text{grad}(x_1, w) = 2$
(Vì $x_1 = 2 \times w$, w thay đổi
1 thì x_1 thay đổi 2)

Bước 3: Áp dụng Quy tắc Chuỗi



Nhân các **đạo hàm cục bộ** dọc theo **đường** đi từ Loss về **Trọng số** để có đạo hàm toàn cục.

Đạo hàm của w :
$$\text{grad}(\text{loss_val}, w) = 1 \times 1 \times 2 = 2$$

Đạo hàm của b :
$$\text{grad}(\text{loss_val}, b) = 1 \times 1 = 1$$

Ý nghĩa: Tăng w thêm 1 đơn vị sẽ làm **Sai số tăng thêm 2 đơn vị. Ta phải làm ngược lại!**

Ma trận Tổng hợp: Hai Chiều Của Mạng

	Lan truyền xuôi	Lan truyền ngược
Hướng đi	Từ Đầu vào -> Đầu ra	Từ Đầu ra -> Đầu vào
Luồng Dữ liệu	Tính toán Tensor thông thường	Chuyển tiếp Đạo hàm (Gradients)
Nền tảng Toán học	Biến đổi Hình học	Áp dụng Quy tắc Chuỗi (Chain Rule)
Mục đích	Đo lường độ chính xác (Loss)	Cung cấp tín hiệu để học hỏi

Đóng vòng lặp: Cập nhật Trọng số



Công thức Tối ưu:

$$W_{\text{mới}} = W_{\text{cũ}} - (\text{Learning_Rate} \times \text{Gradient})$$

Với Learning Rate = 0.1:

$$w_{\text{mới}} = 3 - (0.1 \times 2) = 2.8$$

$$b_{\text{mới}} = 1 - (0.1 \times 1) = 0.9$$



Chạy lại đồ thị với **Trọng số mới** này, sai số (Loss) chắc chắn sẽ **giảm xuống!**

Kỷ nguyên Vi phân Tự động (Auto-diff)



Thực tế

Nhờ các Đồ thị tính toán, bạn không bao giờ phải tự tính đạo hàm bằng tay như vừa rồi.

Sức mạnh Framework

Các công cụ như **TensorFlow** (thông qua **tf.GradientTape**) tự động xây dựng Đồ thị tính toán ngay trong quá trình Forward Pass.

Hậu trường

Khi gọi lệnh tính Gradient, framework tự động duyệt ngược đồ thị và tính toán mọi giá trị cho bạn.

Tổng kết: Bản chất của sự Học hỏi



Deep Learning là Biến đổi Hình học.

Mạng nơ-ron là một chuỗi các phép toán tensor có thể vi phân được.



Phép tính là Dữ liệu.

Đồ thị tính toán là cấu trúc dữ liệu thiên tài cho phép máy tính tự động theo dõi phức tạp toán học.



Backprop là Quy tắc Chuỗi.

Thuật toán Lan truyền ngược chỉ đơn giản là áp dụng Quy tắc chuỗi lên Đồ thị tính toán, lùi từ kết quả Loss về tham số để tối ưu hóa.